Inside the proofs

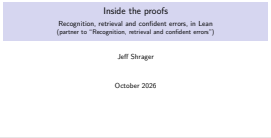
Recognition, retrieval and confident errors, in Lean

(partner to "Recognition, retrieval and confident errors")

Jeff Shrager

October 2026

Inside the proofs



This talk walks through the actual Lean code behind the talk on recognition, retrieval and confident errors. Each code slide is preceded by a slide that says the same thing in English, so you can follow the argument without reading Lean; read in order, the English slides give the whole proof. The point is architecture: how one behavioral theorem is assembled from small, general pieces, and then inherited.

The proof, layer by layer

`MultiDigit/AnyMemory`, `Experiments/FaceNaming`: instances

`Phenomena/RetrievalLaws`: laws; `confident_errors_of_laws`

`Process/RecognizeRetrieve`: the composition; `confident_errors`

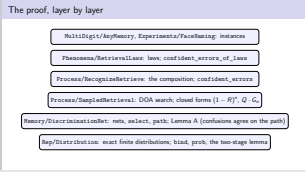
`Process/SampledRetrieval`: DOA search; closed forms $(1 - R)^n$, $Q \cdot G_n$

`Memory/DiscriminationNet`: nets, `select`, `path`; Lemma A (confusions agree on the path)

`Rep/Distribution`: exact finite distributions; `bind`, `prob`, the two-stage lemma

Inside the proofs

The proof, layer by layer



Here's the proof layer by layer, from the bottom. Exact finite distributions and the two-stage lemma. Discrimination nets and Lemma A. The search process from the distributions-of-associations model, with its closed forms. The composition of recognition and retrieval, where confident errors is proved. The laws, from which its hypotheses follow. And the instances, multi-digit addition and face naming. Each layer imports only the ones below it, and the bottom five are the shared library, which imports no theory.

In words: Exact distributions

- A **distribution** is a finite list of outcomes, each with an exact rational probability.
- **Certain**: the distribution certain of *a* has the single outcome *a*, with probability 1.
- **Two stages** (*bind*): draw *a* from the first distribution, then draw from a second distribution that depends on *a*. Each second-stage probability is multiplied by the probability of the *a* that led to it.
- **Probability of an event**: add up the probabilities of the outcomes in the event.

In words: Exact distributions

- A **distribution** is a finite list of outcomes, each with an exact rational probability.
- **Certain**: the distribution certain of a has the single outcome a, with probability 1.
- **Two stages** (*bind*): draw a from the first distribution, then draw from a second distribution that depends on a. Each second-stage probability is multiplied by the probability of the a that led to it.
- **Probability of an event**: add up the probabilities of the outcomes in the event.

Everything rests on exact probability distributions. A distribution is a finite list of outcomes, each with an exact rational probability. The certain distribution has one outcome with probability one. A two-stage process draws a first outcome, then draws from a second distribution that depends on it, multiplying the probabilities. And the probability of an event adds up the probabilities of its outcomes.

Exact distributions

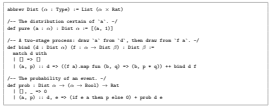
```
abbrev Dist (α : Type) := List (α × Rat)

/-- The distribution certain of `a`. -/
def pure (a : α) : Dist α := [(a, 1)]

/-- A two-stage process: draw `a` from `d`, then draw from `f a`. -/
def bind (d : Dist α) (f : α → Dist β) : Dist β :=
  match d with
  | [] => []
  | (a, p) :: d => ((f a).map fun (b, q) => (b, p * q)) ++ bind d f

/-- The probability of an event. -/
def prob : Dist α → (α → Bool) → Rat
  | [], _ => 0
  | (a, p) :: d, e => (if e a then p else 0) + prob d e
```

Exact distributions



```
abbrev Dist (α : Type) := List (α × Rat)

/-- The distribution certain of `a`. -/
def pure (a : α) : Dist α := [(a, 1)]

/-- A two-stage process: draws `a` from `d`, then draw from `f a`. -/
def bind (d : Dist α) (f : α → Dist β) : Dist β :=
  match d with
  | [] => []
  | (a, p) :: d => ((f a).map fun (b, q) => (b, p * q)) ++ bind d f

/-- The probability of an event. -/
def prob : Dist α → (α → Bool) → Rat
  | [], _ => 0
  | (a, p) :: d, e => (if e a then p else 0) + prob d e
```

And here are those definitions in Lean. A distribution is literally a list of pairs of an outcome and a rational number. Bind, the two-stage process, multiplies each second-stage probability by the first-stage one and appends the results. There's no floating point, no measure theory, and no Mathlib. Because it's all computable, the same definitions are both run and proved about.

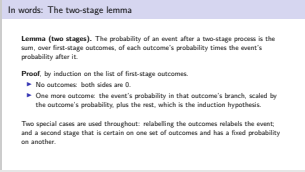
In words: The two-stage lemma

Lemma (two stages). The probability of an event after a two-stage process is the sum, over first-stage outcomes, of each outcome's probability times the event's probability after it.

Proof, by induction on the list of first-stage outcomes.

- No outcomes: both sides are 0.
- One more outcome: the event's probability in that outcome's branch, scaled by the outcome's probability, plus the rest, which is the induction hypothesis.

Two special cases are used throughout: relabelling the outcomes relabels the event; and a second stage that is certain on one set of outcomes and has a fixed probability on another.



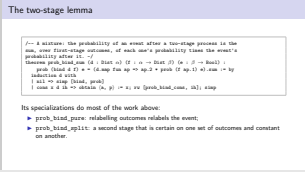
The first lemma says that the probability of an event after a two-stage process is a weighted average: for each first-stage outcome, its probability times the event's probability after it. The proof is an induction on the first-stage list: empty gives zero on both sides, and adding an outcome adds its weighted branch to the rest. Two special cases are used everywhere later: relabelling outcomes, and a second stage that is certain on one set and constant on another.

The two-stage lemma

```
/-- A mixture: the probability of an event after a two-stage process is the
sum, over first-stage outcomes, of each one's probability times the event's
probability after it. -/
theorem prob_bind_sum (d : Dist α) (f : α → Dist β) (e : β → Bool) :
    prob (bind d f) e = (d.map fun ap => ap.2 * prob (f ap.1) e).sum := by
  induction d with
  | nil => simp [bind, prob]
  | cons x d ih => obtain ⟨a, p⟩ := x; rw [prob_bind_cons, ih]; simp
```

Its specializations do most of the work above:

- `prob_bind_pure`: relabelling outcomes relabels the event;
- `prob_bind_split`: a second stage that is certain on one set of outcomes and constant on another.



In Lean, that's three lines of proof: induction on the list, the empty case by simplification, and the other case by the one-step lemma and the induction hypothesis. Almost every probabilistic argument in the rest of the talk reduces to this lemma or one of its two specializations.

In words: Discrimination nets

- A **net** is either a terminal, holding a stored image or nothing, or a test on one feature with two subnets.
- **Selecting** a code: at each test, follow the branch the code's feature says; return the image at the terminal reached.
- The **path** of a code: the features tested on the way down.
- A net **recognizes** a code when the code selects its own image.

In words: Discrimination nets

- A **net** is either a terminal, holding a stored image or nothing, or a test on one feature with two subnets.
- **Selecting** a code: at each test, follow the branch the code's feature says; return the image at the terminal reached.
- The **path** of a code: the features tested on the way down.
- A net **recognizes** a code when the code selects its own image.

Next, EPAM's discrimination nets. A net is either a terminal, which holds a stored image or nothing, or a test on one feature with two subnets. To select a code, follow the branch its feature says at each test and return the image you reach. The path is the list of features tested on the way. And a net recognizes a code when the code selects its own image.

Discrimination nets

```
inductive Net where
  | leaf : Option Item → Net
  | test : Feature → Net → Net → Net

/-- Selection: sort a code to a terminal and return its image. -/
def select : Net → Item → Option Item
  | leaf img, _ => img
  | test f n0 n1, x => if x.feat f then n1.select x else n0.select x

/-- The features examined while sorting `x`: the path it takes. -/
def path : Net → Item → List Feature
  | leaf _, _ => []
  | test f n0 n1, x => f :: (if x.feat f then n1.path x else n0.path x)
```

Inside the proofs

Discrimination nets



In Lean, the net is an inductive type with two constructors, leaf and test, and select and path are defined by recursion on it, exactly as described. Learning, which isn't shown here, replaces a confused terminal with a test on the first noticed difference.

In words: Lemma A

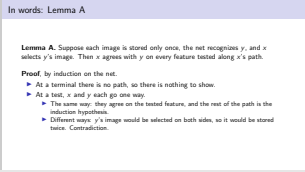
Lemma A. Suppose each image is stored only once, the net recognizes y, and x selects y's image. Then x agrees with y on every feature tested along x's path.

Proof, by induction on the net.

- At a terminal there is no path, so there is nothing to show.
- At a test, x and y each go one way.
  - The same way: they agree on the tested feature, and the rest of the path is the induction hypothesis.
  - Different ways: y's image would be selected on both sides, so it would be stored twice. Contradiction.

Inside the proofs

In words: Lemma A



Lemma A is the key fact about confusions. If each image is stored only once, the net recognizes y, and x selects y's image, then x agrees with y on every feature tested along x's path. The proof is an induction on the net. At a terminal there's no path. At a test, if x and y go the same way they agree on that feature, and the induction hypothesis handles the rest; if they went different ways, y's image would be reachable on both sides, stored twice, which is a contradiction.

Lemma A: a confusion agrees on the path

```
theorem confusion_agrees_on_path (N : Net) (x y : Item) (hnd : N.images.Nodup)
    (hy : N.select y = some y) (hx : N.select x = some y) :
    ∀ f ∈ N.path x, x.feat f = y.feat f := by
  induction N with
  | leaf img => simp [path]
  | test f n0 n1 ih0 ih1 =>
    -- ...
    cases hxf : x.feat f <;> cases hyf : y.feat f <;> simp [hxf, hyf] at hx hy hg
    · -- both go to n0: the tested feature agrees, recurse
      rcases hg with rfl | hg
      · rw [hxf, hyf]
      · exact ih0 hnd0 hy hx g hg
    · -- x to n0, y to n1: y would be stored in both subtrees
      exact absurd rfl (hdisj y (select_mem_images n0 x y hx) y (select_mem_images n1 y y
      ↪ hy))
    -- ... (the two symmetric cases)
```

Lemma A: a confusion agrees on the path

```
theorem confusion_agrees_on_path (H : Set) (x y : Item) (hxd : H.images.Nodup)
    (hy : H.select y = some y) (hx : H.select x = some x) :
    ∀ f ∈ H.path x, x.feat f = y.feat f := by
induction H with
| leaf lag => simp [path]
| test f n0 n1 ih0 ih1 =>
  -- ...
  cases hxf : x.feat f <;> cases hyf : y.feat f <;> simp [hxf, hyf] at hx hy hg
  -- ... (three cases: the tested feature agrees; recurse
    · exact hg with ih1 | hg
      rw [hxf, hyf]
    · exact ih0 hxd0 hy hx g hg
  -- x to n0, y to n1: y would be stored in both subtrees
    exact absurd rfl (hxd.1 y (select_mem_images n0 x y hx) y (select_mem_images n1 y y
      · hy))
  -- ... (the two symmetric cases)
```

Here is the Lean, abbreviated. It's a structural induction: at a test, it cases on the tested feature for x and for y, which gives four cases. In the two where they go the same way, the tested feature agrees and the induction hypothesis covers the rest of the path. In the two where they diverge, the disjointness of the subtrees' images gives a contradiction. The hypothesis that each image is stored once is exactly what EPAM's learning guarantees.

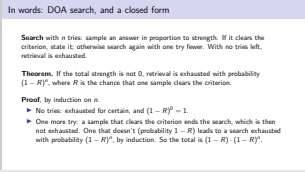
In words: DOA search, and a closed form

Search with n tries: sample an answer in proportion to strength. If it clears the criterion, state it; otherwise search again with one try fewer. With no tries left, retrieval is exhausted.

Theorem. If the total strength is not 0, retrieval is exhausted with probability $(1 - R)^n$, where R is the chance that one sample clears the criterion.

Proof, by induction on n.

- No tries: exhausted for certain, and $(1 - R)^0 = 1$.
- One more try: a sample that clears the criterion ends the search, which is then not exhausted. One that doesn't (probability $1 - R$) leads to a search exhausted with probability $(1 - R)^n$, by induction. So the total is $(1 - R) \cdot (1 - R)^n$.



Now the search from the distributions-of-associations model. With n tries, sample an answer in proportion to strength, state it if it clears the criterion, and otherwise try again; with no tries left, retrieval is exhausted. The theorem: if the total strength isn't zero, retrieval is exhausted with probability one minus R to the n, where R is the chance that one sample clears the criterion. By induction on n: a successful sample ends the search, and an unsuccessful one, with probability one minus R, leads to a search that is exhausted with probability one minus R to the n.

DOA search, and a closed form

```
def search (as : List α) (s : α → Rat) (cc : Rat) : Nat → Nat → Dist (Retrieval α)
  | _, 0 => Dist.pure .exhausted
  | k, n + 1 => Dist.bind (sample as s) fun a =>
      if above s cc a then Dist.pure (.stated a (k + 1)) else search as s cc (k + 1) n

theorem prob_exhausted (as : List α) (s : α → Rat) (cc : Rat) (hT : total as s ≠ 0) :
    ∀ n k, prob (search as s cc k n) Retrieval.isExhausted = (1 - reach as s cc) ^ n := by
  intro n
  induction n with
  | zero => intro k; simp [search, prob_pure, Retrieval.isExhausted]
  | succ n ih =>
    intro k
    simp only [search]
    rw [prob_bind_split _ _ _ (fun _ => false) (fun a => !above s cc a) ((1 - reach as s
    ↪  cc) ^ n)]
    · rw [prob_false, prob_not_above as s cc hT, Rat.pow_succ]; grind
    · intro a
      by_cases ha : above s cc a
      · simp [ha, prob_pure, Retrieval.isExhausted]; grind
      · simp [ha, ih]; grind
```

DOA search, and a closed form

```
def search (ss : List ℕ) (s : ℕ → Bat) (cc : Bat) : Bat → Bat → Dist (Retrieval ℕ)
  | _, 0 => Dist.pure .exhausted
  | k, n + 1 => Dist.bind (sample ss s) fun a =>
      if above a cc a then Dist.pure ⟨.stated a (k + 1)⟩ else search ss s cc (k + 1) n

theorem prob_exhausted (ss : List ℕ) (s : ℕ → Bat) (cc : Bat) (hT : total ss s cc 0) :
    ∀ k, prob (search ss s cc k k) Retrieval.isExhausted = (1 - reach ss s cc) ^ k := by
  intro k
  induction k with
  | zero => intro h; simp [search, prob_pure, Retrieval.isExhausted]
  | succ n ih =>
    intro h
    simp only [search]
    rw [prob_bind_split _ _ _ (fun _ => false) (fun a => !above a cc a) ((1 - reach ss s
      cc) ^ n)]
    · rw [prob_false, prob_not_above ss s cc hT, Nat.pow_succ]; grind
    · intro a ha
      by_cases ha : above a cc a
      · simp [ha, prob_pure, Retrieval.isExhausted]; grind
      · simp [ha, ih]; grind
```

The Lean definition of search is a recursive distribution, and the closed form is proved by induction on the number of tries, splitting each step with the second special case of the two-stage lemma. Notice the hypothesis that the total strength isn't zero. It's real: with zero total, sampling divides by zero, and the blind simulation found that this excludes a case the model needs.

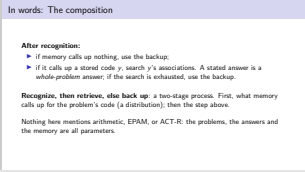
In words: The composition

After recognition:

- if memory calls up nothing, use the backup;
- if it calls up a stored code y, search y's associations. A stated answer is a *whole-problem* answer; if the search is exhausted, use the backup.

Recognize, then retrieve, else back up: a two-stage process. First, what memory calls up for the problem's code (a distribution); then the step above.

Nothing here mentions arithmetic, EPAM, or ACT-R: the problems, the answers and the memory are all parameters.



Now the composition. After recognition, if memory calls up nothing, use the backup; if it calls up a stored code, search that code's associations, and if the search is exhausted, use the backup. The whole trial is a two-stage process: first what memory calls up for the problem's code, then that step. Nothing in it mentions arithmetic, EPAM or ACT-R; the problems, the answers and the memory are all parameters.

The composition

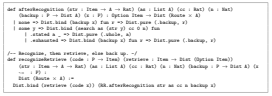
```
def afterRecognition (str : Item → A → Rat) (as : List A) (cc : Rat) (n : Nat)
    (backup : P → Dist A) (x : P) : Option Item → Dist (Route × A)
  | none => Dist.bind (backup x) fun r => Dist.pure (.backup, r)
  | some y => Dist.bind (search as (str y) cc 0 n) fun
      | .stated a _ => Dist.pure (.whole, a)
      | .exhausted => Dist.bind (backup x) fun r => Dist.pure (.backup, r)

/-- Recognize, then retrieve, else back up. -/
def recognizeRetrieve (code : P → Item) (retrieve : Item → Dist (Option Item))
    (str : Item → A → Rat) (as : List A) (cc : Rat) (n : Nat) (backup : P → Dist A) (x
    ↪  : P) :
    Dist (Route × A) :=
  Dist.bind (retrieve (code x)) (RR.afterRecognition str as cc n backup x)
```

Generic in the problem type `P`, the answer type `A`, and the memory (`retrieve`).

Inside the proofs

The composition



```
def afterRecognition (ctr : Item → A → Nat) (as : List A) (cc : Nat) (a : Nat)
    (backup : P → Dist A) (s : P) : Option Item → Dist (Knots × A)
  | none => Dist.bind (backup s) fun x => Dist.pure (.backup, x)
  | some y => Dist.bind (search as (ctr y) cc 0 a) fun
      | .stacked a _ => Dist.pure (.shale, a)
      | .exhausted => Dist.bind (backup s) fun x => Dist.pure (.backup, x)

/-- Recognize, then retrieve, else back up. -/
def recognizeRetrieve (code : P → Item) (retrieve : Item → Dist (Option Item))
    (ctr : Item → A → Nat) (as : List A) (cc : Nat) (a : Nat) (backup : P → Dist A) (s
      : P) :
    Dist (Knots × A) :=
  Dist.bind (retrieve (code s)) (RNA.afterRecognition ctr as cc a backup s)
```

Generic in the problem type `P`, the answer type `A`, and the memory (`retrieve`).

In Lean, after-recognition is a function of what memory called up, and the composition is a single bind. It's generic in the problem type, the answer type and the memory, which is any function from a probe to a distribution over stored codes.

In words: Confident errors: the proof is a composition

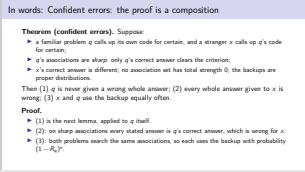
Theorem (confident errors). Suppose:

- a familiar problem q calls up its own code for certain, and a stranger x calls up q's code for certain;
- q's associations are *sharp*: only q's correct answer clears the criterion;
- x's correct answer is different; no association set has total strength 0; the backups are proper distributions.

Then (1) q is never given a wrong whole answer; (2) every whole answer given to x is wrong; (3) x and q use the backup equally often.

Proof.

- (1) is the next lemma, applied to q itself.
- (2): on sharp associations every stated answer is q's correct answer, which is wrong for x.
- (3): both problems search the same associations, so each uses the backup with probability $(1 - R_q)^n$.



Here is the theorem this talk is about. Suppose a familiar problem q calls up its own code for certain, a stranger x calls up q's code for certain, q's associations are sharp, and x's correct answer differs. Then q is never given a wrong whole answer, every whole answer given to x is wrong, and x and q use the backup equally often. The proof has three parts, each a lemma: the neighbour's-answer lemma applied to q itself; the fact that on sharp associations every stated answer is q's correct answer, which is wrong for x; and equal backup use, because both search the same associations.

Confident errors: the proof is a composition

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (recognizeRetrieve code retrieve str as cc n backup)
      (wholeWrong correct) isWhole isBackup x q := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup correct q q
  ↪  (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup correct x _ _ (hT _) hx hs
    ↪  hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]

def ConfusionErrorPattern (trial : P → Dist O) (wholeWrong : P → O → Bool)
    (isWhole isBackup : O → Bool) (x q : P) : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧
    prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧
    prob (trial x) isBackup = prob (trial q) isBackup
```

Confident errors: the proof is a composition



```
theorem confident_errors (correct : P → A) (x q : P) (hf : ∀ y, total as (str y) ≠ x)
    (hbs : mass (backup x) < 1) (hbq : mass (backup q) = 1)
    (hq : retrieves (code q) = pure (some (code q)))
    (hx : retrieves (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (trivialAccessRetrieve code retrieve str as cc a backup)
      (wholeAnswer correct) (wholeAnswer q) x q := by
  refine ⟨?_, ?_, ?_⟩
  · exact (confusion_gives_neighbours_answer code retrieve str as cc a backup correct q q
      (by rfl) (by rfl)
      (wrong_eq_one_of_sharp code retrieve str as cc a backup correct x _ (hf _) hs hne
        ▸ hbs), ?_⟩
  · rw [backup_prob_certain code retrieve str as cc a backup x _ hx hbs hf,
      backup_prob_certain code retrieve str as cc a backup q _ hq hbq hf]

def ConfusionErrorPattern (trial : P → Dist Q) (wholeWrong : P → Q → Real)
    (isWhole isBackup : Q → Real) (x q : P) : Prop :=
  prob (trial q) (wholeWrong q) > 0 ∧
    prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧
    prob (trial x) isBackup = prob (trial q) isBackup
```

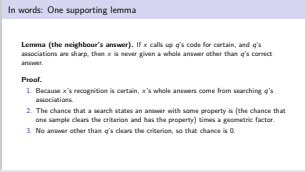
And here is the actual Lean. You don't need to read it in detail. The statement at the top is the theorem I just described. Its conclusion names a phenomenon, the confusion-error pattern, whose definition is at the bottom: three equations about observable outcomes, with no mechanism in them. The proof is the four lines in between, which assemble those three lemmas. It's that short because the real work sits in general lemmas underneath: about probability distributions, about the search process, about nets. That's the point I want to make with this slide. In a library, a behavioral theorem is assembled from reusable parts, the way Mathlib assembles algebra.

In words: One supporting lemma

Lemma (the neighbour's answer). If x calls up q's code for certain, and q's associations are sharp, then x is never given a whole answer other than q's correct answer.

Proof.

1. Because x's recognition is certain, x's whole answers come from searching q's associations.
2. The chance that a search states an answer with some property is (the chance that one sample clears the criterion and has the property) times a geometric factor.
3. No answer other than q's clears the criterion, so that chance is 0.



The neighbour's-answer lemma: if x calls up q's code for certain and q's associations are sharp, x is never given a whole answer other than q's correct answer. First, since x's recognition is certain, x's whole answers come from searching q's associations. Second, the chance of stating an answer with some property is the chance that one sample clears the criterion and has the property, times a geometric factor. Third, no answer other than q's clears the criterion, so that chance is zero.

One supporting lemma

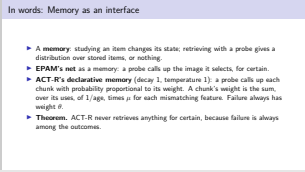
```
theorem confusion_gives_neighbours_answer (correct : P → A) (x q : P)
    (hT : total as (str (code q)) ≠ 0) (h : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) :
    prob (recognizeRetrieve code retrieve str as cc n backup x)
      (wholeSuch fun a => a != correct q) = 0 := by
  rw [whole_depends_only_on_route code retrieve str as cc n backup x _ _ h,
    prob_stated _ _ _ _ hT]
  have : reachSuch as (str (code q)) cc (fun a => a != correct q) = 0 := by
    simp only [reachSuch]
    rw [prob_congr _ _ (fun _ => false), prob_false]
    intro a
    by_cases ha : above (str (code q)) cc a
    · simp [hs a ha]
    · simp [ha]
  rw [this]; grind
```

```
theorem confusion_gives_neighbours_answer {correct : P → A} (x q : P)
    (hd : ∃ a, a ≠ correct q ∧ μ {y | retrieve (code x) = pure (some (code q))
                                    ∧ retrieve (code y) (correct q) = pure a} ≠ 0) :
    prob (recognisesRetrieve code retrieve str ac cc a backup x)
      (shellReach fun a => a ≠ correct q) = 0 := by
rw [whole_depends_only_on_route code retrieve str ac cc a backup x _ _ h,
  prob_stated _ _ _ hT]
have : reachBuch ac (str (code q)) cc (fun a => a ≠ correct q) = 0 := by
  simp only [reachBuch]
  rw [prob_congr _ _ (fun _ => false), prob_false]
  intro a
  by_cases ha : above (str (code q)) cc a
  · simp [ha = ha]
  · simp [ha]
rw [this]; grind
```

In Lean, those three steps are two rewrites and a pointwise argument. The first rewrite says whole-problem retrieval on x is retrieval on q's associations; the second applies the closed form for stating an answer. The remaining goal, that the chance of reaching an answer other than q's is zero, is proved one answer at a time, using sharpness.

In words: Memory as an interface

- A **memory**: studying an item changes its state; retrieving with a probe gives a distribution over stored items, or nothing.
- **EPAM's net** as a memory: a probe calls up the image it selects, for certain.
- **ACT-R's declarative memory** (decay 1, temperature 1): a probe calls up each chunk with probability proportional to its weight. A chunk's weight is the sum, over its uses, of $1/\text{age}$, times μ for each mismatching feature. Failure always has weight θ.
- **Theorem.** ACT-R never retrieves anything for certain, because failure is always among the outcomes.



Memory is an interface: studying changes the state, and retrieving with a probe gives a distribution over stored items. EPAM's net retrieves the image it selects, for certain. ACT-R's declarative memory, with decay one and temperature one, retrieves each chunk in proportion to its weight, which is the sum over its uses of one over age, times mu for each mismatching feature, with failure always possible. And so a theorem: ACT-R never retrieves anything for certain.

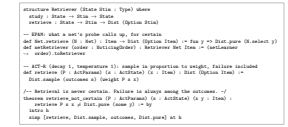
Memory as an interface

```
structure Retriever (State Stim : Type) where
  study : State → Stim → State
  retrieve : State → Stim → Dist (Option Stim)

-- EPAM: what a net's probe calls up, for certain
def Net.retrieve (N : Net) : Item → Dist (Option Item) := fun y => Dist.pure (N.select y)
def netRetriever (order : NoticingOrder) : Retriever Net Item := (netLearner
↪  order).toRetriever

-- ACT-R (decay 1, temperature 1): sample in proportion to weight, failure included
def retrieve (P : ActParams) (s : ActState) (x : Item) : Dist (Option Item) :=
  Dist.sample (outcomes s) (weight P s x)

/-- Retrieval is never certain. Failure is always among the outcomes. -/
theorem retrieve_not_certain (P : ActParams) (s : ActState) (x y : Item) :
    retrieve P s x ≠ Dist.pure (some y) := by
  intro h
  simp [retrieve, Dist.sample, outcomes, Dist.pure] at h
```



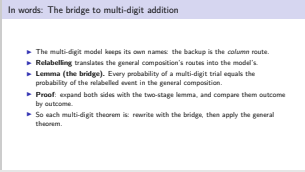
In Lean, a retriever is a structure with two fields, study and retrieve. EPAM's retrieval wraps the net's selection in a certain distribution; ACT-R's is a sample over failure and the stored chunks. The proof that ACT-R is never certain is a single simplification: its list of outcomes begins with failure, so it can never equal the certain distribution on a chunk.

In words: The bridge to multi-digit addition

- The multi-digit model keeps its own names: the backup is the *column* route.
- **Relabelling** translates the general composition's routes into the model's.
- **Lemma (the bridge).** Every probability of a multi-digit trial equals the probability of the relabelled event in the general composition.
- **Proof**: expand both sides with the two-stage lemma, and compare them outcome by outcome.
- So each multi-digit theorem is: rewrite with the bridge, then apply the general theorem.

Inside the proofs

In words: The bridge to multi-digit addition



The multi-digit model keeps its own names, calling the backup the column route. A relabelling translates the general composition's routes into its names, and a bridge lemma says every probability of a multi-digit trial equals the probability of the relabelled event in the general composition: expand both sides with the two-stage lemma and compare them outcome by outcome. Then each multi-digit theorem is a rewrite with the bridge followed by the general theorem.

The bridge to multi-digit addition

```
/-- The generic routes, in this model's names. -/
def relabel : Cog.Route × Nat → Route × Nat
  | (.whole, a) => (.whole, a)
  | (.backup, a) => (.columns, a)

theorem prob_trialWith (x : Problem) (e : Route × Nat → Bool) :
    prob (trialWith retrieve E str as cc n backup x) e =
      prob (recognizeRetrieve E.code retrieve str as cc n backup x) (fun o => e (relabel
      ↪  o)) := by
  unfold trialWith recognizeRetrieve
  rw [prob_bind_sum, prob_bind_sum]
  congr 1
  apply List.map_congr_left
  intro rp _
  rw [prob_after]

-- then each multi-digit theorem is two lines:
theorem confident_errors ... := by
  simp only [prob_trialWith, wholeWrong_relabel, isWhole_relabel, isColumns_relabel]
  exact RR.confident_errors E.code retrieve str as cc n backup Problem.sum x q ...
```

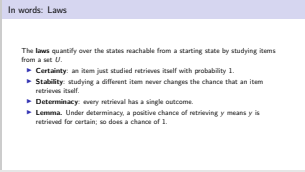


Here's the relabelling, the bridge lemma and its proof, which unfolds both definitions, applies the two-stage lemma on each side and compares them pointwise. At the bottom you can see the multi-digit confident-errors theorem: rewrite with the bridge and the event identities, then apply the general theorem. Keeping the model's own definitions meant its exported numbers didn't change at all.

In words: Laws

The **laws** quantify over the states reachable from a starting state by studying items from a set U.

- **Certainty**: an item just studied retrieves itself with probability 1.
- **Stability**: studying a different item never changes the chance that an item retrieves itself.
- **Determinacy**: every retrieval has a single outcome.
- **Lemma.** Under determinacy, a positive chance of retrieving y means y is retrieved for certain; so does a chance of 1.



The laws quantify over states reachable from a starting state by studying items from a set. Certainty: an item just studied retrieves itself with probability one. Stability: studying a different item never changes the chance that an item retrieves itself. Determinacy: every retrieval has a single outcome. And a lemma: under determinacy, a positive chance of retrieving y means that y is retrieved for certain.

Laws

```
def Certainty (R : Retriever S Item) (s0 : S) (U : List Item) : Prop :=
  ∀ s, Reachable R s0 U s → ∀ y ∈ U, pRet R (R.study s y) y y = 1

def Stability (R : Retriever S Item) (s0 : S) (U : List Item) : Prop :=
  ∀ s, Reachable R s0 U s → ∀ y ∈ U, ∀ z ∈ U, z ≠ y → pRet R (R.study s z) y y = pRet R
  ↪  s y y

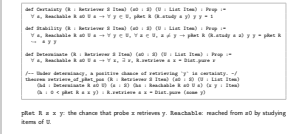
def Determinate (R : Retriever S Item) (s0 : S) (U : List Item) : Prop :=
  ∀ s, Reachable R s0 U s → ∀ x, ∃ r, R.retrieve s x = Dist.pure r

/-- Under determinacy, a positive chance of retrieving `y` is certainty. -/
theorem retrieve_of_pRet_pos (R : Retriever S Item) (s0 : S) (U : List Item)
    (hd : Determinate R s0 U) (s : S) (hs : Reachable R s0 U s) (x y : Item)
    (h : 0 < pRet R s x y) : R.retrieve s x = Dist.pure (some y)
```

`pRet R s x y`: the chance that probe `x` retrieves `y`. `Reachable`: reached from `s0` by studying items of `U`.

Inside the proofs

Laws



In Lean, each law is a definition of a proposition over the retriever interface, with no mechanism mentioned. Each cell of the matrix is then a theorem that a particular family's retriever has the law, or that it doesn't.

In words: Studied items retrieve themselves

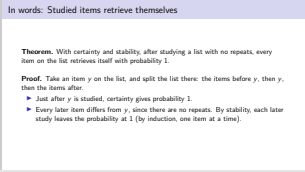
Theorem. With certainty and stability, after studying a list with no repeats, every item on the list retrieves itself with probability 1.

Proof. Take an item y on the list, and split the list there: the items before y, then y, then the items after.

- Just after y is studied, certainty gives probability 1.
- Every later item differs from y, since there are no repeats. By stability, each later study leaves the probability at 1 (by induction, one item at a time).

Inside the proofs

In words: Studied items retrieve themselves



Next, a theorem: with certainty and stability, after studying a list with no repeats, every item on the list retrieves itself with probability one. Split the list at the item: just after it's studied, certainty gives probability one; every later item is different, since there are no repeats, and stability, applied one item at a time, keeps the probability at one. This is also the whole content of the flat serial-position curve in our study-test experiment.

Studied items retrieve themselves

```
theorem studied_selfRetrieval (R : Retriever S Item) (s0 : S) (U : List Item)
    (hc : Certainty R s0 U) (hs : Stability R s0 U) (xs : List Item)
    (hU : ∀ x ∈ xs, x ∈ U) (hnd : xs.Nodup) (y : Item) (hy : y ∈ xs) :
    pRet R (R.studyAll s0 xs) y y = 1 := by
  obtain ⟨pre, post, hsplit⟩ := List.append_of_mem hy    -- xs = pre ++ y :: post
  -- ... y ∉ post, from Nodup
  rw [hsplit, show pre ++ y :: post = (pre ++ [y]) ++ post by simp,
    stable_after R s0 U hs y hyU post (pre ++ [y]) _ hpost,   -- studying post changes
    ↪  nothing
    studyAll_append]
  exact hc _ ⟨pre, hpre, rfl⟩ y hyU                          -- certainty, just after y
```

`stable_after` is an induction on `post`, applying stability once per item.

Studied items retrieve themselves

```
theorem studied_selfRetrieval (R : Retriever S Item) (st : S) (y : List Item)
    (hs : Certainty R st y) (ha : Stability R st y) (xs : List Item)
    (hd : ∀ x ∈ xs, x ∈ y) (hnd : xs.Nodup) (y : Item) (hy : y ∈ xs) :
    y∈ R (R.study(hi) st) xs[ y ]? = 1 := by
  obtain ⟨pre, post, hsplit⟩ := List.append_of_mem hy    -- xs = pre ++ y :: post
  -- ... y ∉ post, from Nodup
  rw [hsplit, show pre ++ y :: post = (pre ++ [y]) ++ post by simp,
    stable_after R st y hs y hy0 post (pre ++ [y]) _ hpost,   -- studying post changes
      nothing
    ← hsplit, append]
  exact hs _ (pre, hpre, rfl) y hy0                          -- certainty, just after y
```

stable_after is an induction on post, applying stability once per item.

The Lean proof splits the list using a library lemma, rewrites the study history as the part up to and including y followed by the rest, applies stability to the rest by an inner induction, and finishes with certainty. The comments on the right mark each step.

In words: Confident errors from laws

Theorem. Any memory with determinacy, certainty and stability produces confident errors. After studying a list with no repeats that includes q, if a stranger x can be taken for q (with positive probability), the three conclusions of confident errors hold.

Proof.

1. The memory's state is reachable, so the laws apply to it.
2. q retrieves itself with probability 1 (the previous theorem), hence for certain (determinacy).
3. x retrieves q with positive probability, hence for certain (determinacy).
4. These are exactly the hypotheses of the confident-errors theorem.

In words: Confident errors from laws

Theorem. Any memory with determinacy, certainty and stability produces confident errors. After studying a list with no repeats that includes q, if a stranger x can be taken for q (with positive probability), the three conclusions of confident errors hold.

Proof.

1. The memory's state is reachable, so the laws apply to it.
2. q retrieves itself with probability 1 (the previous theorem), hence for certain (determinacy).
3. x retrieves q with positive probability, hence for certain (determinacy).
4. These are exactly the hypotheses of the confident-errors theorem.

Now the theorem that connects the two halves. Any memory with determinacy, certainty and stability produces confident errors: after studying a list without repeats that includes q, if a stranger x can be taken for q with positive probability, the three conclusions hold. The memory's state is reachable, so the laws apply. q retrieves itself with probability one by the previous theorem, hence for certain; x retrieves q with positive probability, hence also for certain. Those are exactly the hypotheses of confident errors.

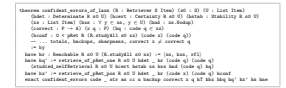
Confident errors from laws

```
theorem confident_errors_of_laws (R : Retriever S Item) (s0 : S) (U : List Item)
    (hdet : Determinate R s0 U) (hcert : Certainty R s0 U) (hstab : Stability R s0 U)
    (xs : List Item) (hxs : ∀ y ∈ xs, y ∈ U) (hnd : xs.Nodup)
    (correct : P → A) (x q : P) (hq : code q ∈ xs)
    (hconf : 0 < pRet R (R.studyAll s0 xs) (code x) (code q))
    -- ... totals, backups, sharpness, correct x ≠ correct q
    := by
  have hr : Reachable R s0 U (R.studyAll s0 xs) := ⟨xs, hxs, rfl⟩
  have hq' := retrieve_of_pRet_one R s0 U hdet _ hr (code q) (code q)
    (studied_selfRetrieval R s0 U hcert hstab xs hxs hnd (code q) hq)
  have hx' := retrieve_of_pRet_pos R s0 U hdet _ hr (code x) (code q) hconf
  exact confident_errors code _ str as cc n backup correct x q hT hbx hbq hq' hx' hs hne
```

Laws supply the two certain-retrieval hypotheses; the composition theorem does the rest.

Inside the proofs

Confident errors from laws

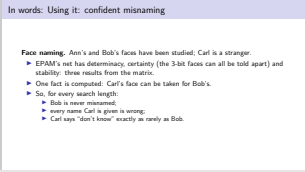


In Lean, the proof is four lines: establish reachability, derive the two certain-retrieval facts from the laws, and apply the confident-errors theorem. What were assumptions about one state are now consequences of three laws and one observable fact.

In words: Using it: confident misnaming

Face naming. Ann's and Bob's faces have been studied; Carl is a stranger.

- EPAM's net has determinacy, certainty (the 3-bit faces can all be told apart) and stability: three results from the matrix.
- One fact is computed: Carl's face can be taken for Bob's.
- So, for every search length:
  - Bob is never misnamed;
  - every name Carl is given is wrong;
  - Carl says "don't know" exactly as rarely as Bob.

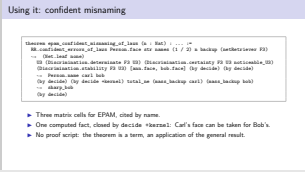


Applying it to faces: EPAM's net, with Ann and Bob studied, has determinacy, certainty, because the three-bit faces can all be told apart, and stability, which are three results from the matrix. One fact is computed: Carl's face can be taken for Bob's. So, for every search length, Bob is never misnamed, every name Carl is given is wrong, and Carl says don't know exactly as rarely as Bob.

Using it: confident misnaming

```
theorem epam_confident_misnaming_of_laws (n : Nat) : ... :=
  RR.confident_errors_of_laws Person.face str names (1 / 2) n backup (netRetriever F3)
  ↪  (Net.leaf none)
    U3 (Discrimination.determinate F3 U3) (Discrimination.certainty F3 U3 noticeable_U3)
    (Discrimination.stability F3 U3) [ann.face, bob.face] (by decide) (by decide)
    ↪  Person.name carl bob
    (by decide) (by decide +kernel) total_ne (mass_backup carl) (mass_backup bob)
    ↪  sharp_bob
    (by decide)
```

- Three matrix cells for EPAM, cited by name.
- One computed fact, closed by `decide +kernel`: Carl's face can be taken for Bob's.
- No proof script: the theorem is a term, an application of the general result.



In Lean, that instance is a single application of the general theorem. It cites EPAM's three matrix cells by name, and the side conditions, such as the faces being distinct and Carl being taken for Bob, are finite checks closed by decide. There's no proof script at all.

In words: Computing inside the proof checker

- Some facts are about particular numbers. For example, with these parameters, $150 + 100$ is answered 200 with probability $45/46$.
- These are proved by **evaluation**: Lean's kernel runs the definitions and checks the resulting equation.
- The same definitions also run as ordinary programs. They produced the exact distributions that the blind simulation was checked against.
- The "lacks" cells of the matrix are proved this way, on small examples.

Inside the proofs

In words: Computing inside the proof checker

In words: Computing inside the proof checker

- Some facts are about particular numbers. For example, with these parameters, 150 + 100 is answered 200 with probability 45/46.
- These are proved by **evaluation**: Lean's kernel runs the definitions and checks the resulting equation.
- The same definitions also run as ordinary programs. They produced the exact distributions that the blind simulation was checked against.
- The "lacks" cells of the matrix are proved this way, on small examples.

Some facts are about particular numbers, for instance that with these parameters 150 plus 100 is answered 200 with probability 45 over 46. These are proved by evaluation: Lean's kernel runs the definitions and checks the resulting equation. The same definitions also run as ordinary programs, and they produced the exact distributions the blind simulation was checked against. The counterexample cells of the matrix are proved this way too.

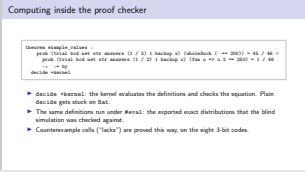
Computing inside the proof checker

```
theorem example_values :
    prob (trial bcd net str answers (1 / 2) 1 backup x) (wholeSuch (· == 200)) = 45 / 46 ∧
      prob (trial bcd net str answers (1 / 2) 1 backup x) (fun o => o.2 == 250) = 1 / 46
      ↪  := by
  decide +kernel
```

- `decide +kernel`: the kernel evaluates the definitions and checks the equation. Plain `decide` gets stuck on `Rat`.
- The same definitions run under `#eval`: the exported exact distributions that the blind simulation was checked against.
- Counterexample cells (“lacks”) are proved this way, on the eight 3-bit codes.

Inside the proofs

Computing inside the proof checker



Here's such a theorem, proved by decide with the kernel: the kernel evaluates both probabilities and checks that they are 45 over 46 and 1 over 46. Plain decide gets stuck on rational numbers, which is why we use the kernel. It's slow, but it's trusted: no compiler or external solver is part of the trusted base.

In words: What is checked, and what is not

- Every theorem is fully proved. Lean reports which axioms a theorem depends on; here, only its three standard ones.
- What proof cannot check is whether the definitions say what EPAM, DOA or ACT-R say. That needs other kinds of evidence: a prose specification, an independent implementation, comparison with community code.

Inside the proofs

In words: What is checked, and what is not

In words: What is checked, and what is not

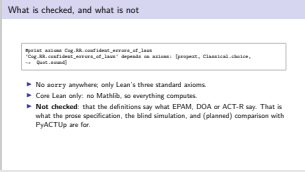
- Every theorem is fully proved. Lean reports which axioms a theorem depends on; here, only its three standard ones.
- What proof cannot check is whether the definitions say what EPAM, DOA or ACT-R say. That needs other kinds of evidence: a prose specification, an independent implementation, comparison with community code.

Finally, what is and isn't checked. Every theorem is fully proved, and Lean reports exactly which axioms each one depends on: here, only its three standard ones. What proof can't check is whether the definitions say what EPAM, the associations model or ACT-R say. That needs other evidence, such as a prose specification, an independent implementation, and comparison with community code.

What is checked, and what is not

```
#print axioms Cog.RR.confident_errors_of_laws
'Cog.RR.confident_errors_of_laws' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

- No `sorry` anywhere; only Lean's three standard axioms.
- Core Lean only: no Mathlib, so everything computes.
- **Not checked**: that the definitions say what EPAM, DOA or ACT-R say. That is what the prose specification, the blind simulation, and (planned) comparison with PyACTUp are for.



What does all this guarantee? Lean will tell you exactly which axioms a theorem rests on. Here it's only Lean's three standard ones, with no unproved steps anywhere. We use core Lean without Mathlib, so everything also computes. What Lean can't check is whether the definitions say what EPAM, or the associations model, or ACT-R actually say. That is a different kind of question, and the next part is about how we tested it.